BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

> **Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the Git Hub Repo. Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance measure, Environment, Actuators, Sensors.

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

The Environment (E) is represented by self.width/self.height (grid size), self.walls, self.food_positions, self.opponents, and self.agent_pos. These describe the physical layout and positions in the world.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi-Agent. self.opponents are independently-moving entities that directly reduce the score (−50) on collision, so the agent is not acting alone.

Task 1.2: The Perception Subsystem (get_percept)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete, ordered history of everything an agent has ever perceived, not just its current input. An agent's action can depend on this whole history, not only the latest percept.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Sensors (S). get_percept() is the only interface through which the agent perceives the environment, returning the limited information it's allowed to "see."

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Partially Observable. The agent sees its own and opponents' exact positions, but only gets booleans (smells_food, hit_wall) for its current tile plus a food count — not the full locations of walls or food elsewhere on the grid.

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance measure (P). self.score -= 5 only changes the score, not the physical environment — the wall stays put and the agent doesn't even move.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

Performance must be judged on external outcomes, not internal effort, because rationality is defined by real consequences in the world. An agent could "look busy" internally yet still crash into walls repeatedly, so rewarding effort instead of

Part 2: Guided Modification & Deep Theory Mapping

> Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (__init__)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__` . Populate it with coordinate tuples safely avoiding position `(0, 0)` , walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

Hiding toxic_traps from the sensors makes the environment even more partially observable. Unlike walls (which trigger hit_wall on contact), a hidden trap gives zero percept even when triggered — the agent only sees an unexplained score drop.

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

smells_toxin only fires the instant the agent is already on the trap, so it can't warn in advance. But it lets a memory-based agent record which tiles are hazardous and avoid them next time, improving its expected utility on future visits.

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic exploit is rewarding the action of cleaning (points per Suck) instead of the outcome of a clean floor. An agent could exploit this by dirtying a square just to clean it again and rack up reward without ever making the environment genuinely cleaner.

Finalize and Lock Lab 01 Analytical Evaluation

FACULTY OF COMPUTING